

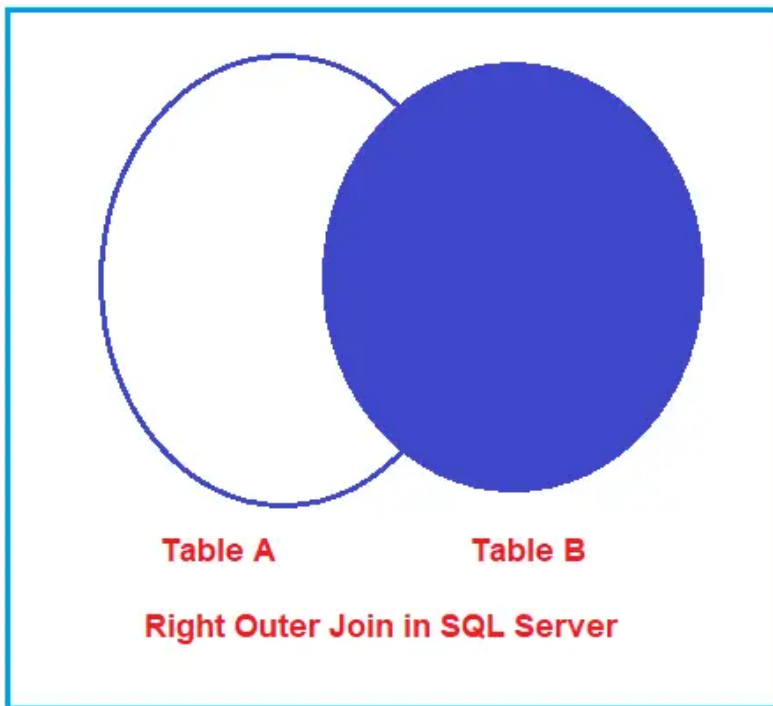
# Right Outer Join in SQL Server with Examples

In this article, I am going to discuss the **Right Outer Join in SQL Server** with Examples. Please read our previous article where we discussed the [SQL Server Left Outer Join](#) with Example. At the end of this article, you will understand the following pointers.

1. [What is Right Outer Join?](#)
2. [How to implement Right Join in SQL Server?](#)
3. [When do we need to use the Right JOIN?](#)
4. [Example to Understand SQL Server Right Outer Join](#)
5. [How to retrieve only the non-matching rows from the right table?](#)
6. [Joining three Tables in SQL Server using Right Join](#)

## What is Right Outer Join in SQL Server?

The Right Outer Join in SQL Server is used to retrieve all the matching records from both the tables involved in the join as well as all the non-matching records from the right-hand side table. In that case, the un-matching data will take the null value. The following diagram shows the pictorial representation of the Right Outer Join in SQL Server.



The question that should come to your mind is which is the left table and which is the right table? The answer is, the table which is mentioned to the left of the RIGHT OUTER JOIN keyword is the left table, and the table which is mentioned to the right of the RIGHT OUTER JOIN keyword is the right table. The following diagram shows the pictorial representation of Right Outer Join.

## How to implement Right Outer Join in SQL Server?

Please have a look at the following image which shows the syntax of Right Outer Join. Here, you can use either the Right Outer Join or Right JOIN keyword to perform the Right Outer Join operation. If you use only

the Right JOIN keyword, then it is also going to perform Right Outer Join in SQL Server.

**Syntax1:**

```
SELECT column_name(s)
FROM table1
RIGHT OUTER JOIN table2
ON table1.column_name = table2.column_name;
```

**Syntax2:**

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

## When do we need to use the Right JOIN in SQL Server?

If you want to select all the records from the Right side table including the records that have a null foreign key value then you need to use the SQL Server Right Outer Join. To simplify the above definition, we can say that if you want to retrieve all the matching rows from both the tables involved in the join as well as the non-matching rows from the right side table then you need to use Right Outer Join in SQL Server.

## Example to Understand SQL Server Right Outer Join.

Let us understand how to implement Right Outer Join in SQL Server with Examples. To understand Right Outer Join, we are going to use the following Company and Candidate tables.

| Company   |             | Candidate |          |           |
|-----------|-------------|-----------|----------|-----------|
| CompanyId | CompanyName | Candidat  | FullName | CompanyId |
| 1         | DELL        | 1         | Ron      | 1         |
| 2         | HP          | 2         | Pete     | 2         |
| 3         | IBM         | 3         | Steve    | 3         |
| 4         | Microsoft   | 4         | Steve    | NULL      |
|           |             | 5         | Ravi     | 1         |
|           |             | 6         | Raj      | 3         |
|           |             | 7         | Kiran    | NULL      |

Please use the following SQL script to create and populate the Company and Candidate tables with some test data. Please note that here the **CompanyId** column of the **Candidate Table** is the foreign key column referencing the **CompanyId** column of the **Company Table**.

```
CREATE TABLE Company
(
    CompanyId TinyInt Identity Primary Key,
```

```

        CompanyName Nvarchar(50) NULL
    )
GO
INSERT Company VALUES('DELL')
INSERT Company VALUES('HP')
INSERT Company VALUES('IBM')
INSERT Company VALUES('Microsoft')
GO
CREATE TABLE Candidate
(
    CandidateId tinyint identity primary key,
    FullName nvarchar(50) NULL,
    CompanyId tinyint REFERENCES Company(CompanyId)
)
GO
INSERT Candidate VALUES('Ron',1)
INSERT Candidate VALUES('Pete',2)
INSERT Candidate VALUES('Steve',3)
INSERT Candidate VALUES('Steve',NULL)
INSERT Candidate VALUES('Ravi',1)
INSERT Candidate VALUES('Raj',3)
INSERT Candidate VALUES('Kiran',NULL)
GO

```

Write a query, to retrieve CandidateId, FullName, CompanyId, and CompanyName from Company and Candidate tables. The output of the query should be as shown below.

| CandidateId | FullName | CompanyId | CompanyId | CompanyName |
|-------------|----------|-----------|-----------|-------------|
| 1           | Ron      | 1         | 1         | DELL        |
| 5           | Ravi     | 1         | 1         | DELL        |
| 2           | Pete     | 2         | 2         | HP          |
| 3           | Steve    | 3         | 3         | IBM         |
| 6           | Raj      | 3         | 3         | IBM         |
| NULL        | NULL     | NULL      | 4         | Microsoft   |

Following is the Query which is an example of Right Outer Join that is joining the Company and Candidate tables and will give the results shown in the above image.

```

SELECT Cand.CandidateId,
       Cand.FullName,
       Cand.CompanyId,
       Comp.CompanyId,
       Comp.CompanyName
FROM Candidate Cand
RIGHT OUTER JOIN Company Comp
ON Cand.CompanyId = Comp.CompanyId

```

If you look at the above, we got 6 rows. We got all the matching rows from both the tables as well as non-matching rows from the right side table i.e. the Company Table. Instead of using the **Right Outer Join** keyword, you can also use the **Right Join** keyword. This will also work as expected as Right Outer Join. That means the SQL Server Left Outer Join or Left Join means the same. The following example uses the Right Join keyword and we also get the same output as the previous example.

```
SELECT Cand.CandidateId,  
       Cand.FullName,  
       Cand.CompanyId,  
       Comp.CompanyId,  
       Comp.CompanyName  
FROM Candidate Cand  
RIGHT JOIN Company Comp  
ON Cand.CompanyId = Comp.CompanyId
```

In short, we can say that Right Outer Join will return all the matching records from both the tables as well as all the non-matching records from the right-hand side table.

## How to retrieve only the non-matching rows from the right table?

Here, what we want is, we want to retrieve only the non-matching records from the right table. For example, we want the following result sets when we join the Candidate and Company tables. If you further notice, here we want to retrieve those rows from the Candidate table whose does not have any reference in the Company table.

| CandidateId | FullName | CompanyId | CompanyId | CompanyName |
|-------------|----------|-----------|-----------|-------------|
| NULL        | NULL     | NULL      | 4         | Microsoft   |

As we already discussed, the Right Outer Join is used to fetch all the matching records from both the tables as well as all the non-matching records from the right-hand side table. In this case, the non-matching records will take NULL value. So, to get the above output what we need to do is, we need to perform the Right Outer Join operation, and then in the where condition we need to filter out the records which have NULL values. The following Right OuterJoin Query with the where clause does the same.

```
SELECT Cand.CandidateId,  
       Cand.FullName,  
       Cand.CompanyId,  
       Comp.CompanyId,  
       Comp.CompanyName  
FROM Candidate Cand  
Right JOIN Company Comp  
ON Cand.CompanyId = Comp.CompanyId  
WHERE Cand.CompanyId IS NULL;
```

## Joining three Tables in SQL Server:

Now we will see how to Right Join three tables in SQL Server. Already we have the Company and Candidate tables. Let us introduce the third table i.e. the Address table. So, we are going to use the following Address table along with the Company and Candidate tables to perform joining operations between three tables.

| AddressId | CandidateId | Country | State       | City    |
|-----------|-------------|---------|-------------|---------|
| 1         | 1           | India   | Odisha      | BBSR    |
| 2         | 2           | India   | Maharashtra | Mumbai  |
| 3         | 3           | India   | Maharashtra | Pune    |
| 4         | 4           | India   | Odisha      | Cuttack |

Please use the following SQL Script to create and populate the Address table with the required test data.

```

CREATE TABLE Address
(
    AddressId INT IDENTITY PRIMARY KEY,
    CandidateId tinyint REFERENCES Company(CompanyId),
    Country VARCHAR(50),
    State VARCHAR(50),
    City VARCHAR(50),
)
GO
INSERT INTO Address Values (1, 'India', 'Odisha', 'BBSR');
INSERT INTO Address Values (2, 'India', 'Maharashtra', 'Mumbai');
INSERT INTO Address Values (3, 'India', 'Maharashtra', 'Pune');
INSERT INTO Address Values (4, 'India', 'Odisha', 'Cuttack');
GO

```

The following is the syntax to join three tables in SQL Server.

```

SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name
RIGHT JOIN table3
ON table1.column_name = table3.column_name;

```

### Example: Right Joining Candidate, Company, and Address tables

The following Right Join SQL Query Joining Candidate, Company, and Address tables.

```

SELECT Cand.CandidateId,
       Cand.FullName,
       Cand.CompanyId,
       Comp.CompanyName,
       Addr.Country,
       Addr.State,
       Addr.City
FROM Candidate Cand
RIGHT JOIN Company Comp

```

```
ON Cand.CompanyId = Comp.CompanyId
RIGHT JOIN Address Addr
ON Addr.CandidateId = Cand.CandidateId;
```

When you execute the above Right Join SQL Query, you will get the following output.

| CandidateId | FullName | CompanyId | CompanyName | Country | State       | City    |
|-------------|----------|-----------|-------------|---------|-------------|---------|
| 1           | Ron      | 1         | DELL        | India   | Odisha      | BBSR    |
| 2           | Pete     | 2         | HP          | India   | Maharashtra | Mumbai  |
| 3           | Steve    | 3         | IBM         | India   | Maharashtra | Pune    |
| NULL        | NULL     | NULL      | NULL        | India   | Odisha      | Cuttack |

In the next article, I am going to discuss [Full Outer Join in SQL Server](#) with Examples. Here, in this article, I try to explain the **SQL Server Right Outer Join** with Examples. I hope this article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.